1 Vorbereitung und Datenbestand

Umgesetzt wurde das ER-Modell der Konzeptionsphase mit zwölf Entitäten und einer Zuordnungstabelle, zusammen dreizehn Tabellen im Schema *buchtausch*. Die SQL-Datei löscht ein vorhandenes Schema *buchtausch* und legt es neu an, daher lässt sie sich beliebig häufig ausführen. Alle Namen, Verlage, Titel und Adressen der Dummy-Daten sind vom Verfasser erfundene Platzhalter. Die Nutzer und ihre Standorte liegen in Berlin, München und Hamburg, damit die Umkreissuche grob über unterschiedliche Entfernungen geprüft werden kann. Jede Tabelle enthält mindestens zehn Zeilen.

SQL-Dokumentation: Schema, Erweiterung und Zeitbereich

```
-- Schema neu anlegen
DROP SCHEMA IF EXISTS buchtausch CASCADE;
-- Schema anlegen
CREATE SCHEMA buchtausch;
-- Folgende Objekte im Schema buchtausch anlegen
SET search_path TO buchtausch;
-- Erweiterung für die EXCLUDE-Constraints
CREATE EXTENSION IF NOT EXISTS btree_gist;
-- Uhrzeiten definieren
CREATE TYPE timerange AS RANGE (subtype = time);
```

Dummy-Daten, Auszug aus der Tabelle Nutzer

```
-- Beispiel Daten für Nutzer
INSERT INTO Nutzer (Name, Vorname, EMail, Telefonnummer) VALUES
    ('Musterfrau', 'Anna',  'anna.musterfrau@example.org', '+49 30 23125001'),
    ('Mustermann', 'Ben',   'ben.mustermann@example.org',  '+49 30 23125002'),
    ('Beispiel',   'Clara', 'clara.beispiel@example.org',  NULL),
    ('Testmann',   'David', 'david.testmann@example.org',  '+49 30 23125004'),
    ...;
```

Zeilenzahl je Tabelle (Abfrage 6 der SQL-Datei)

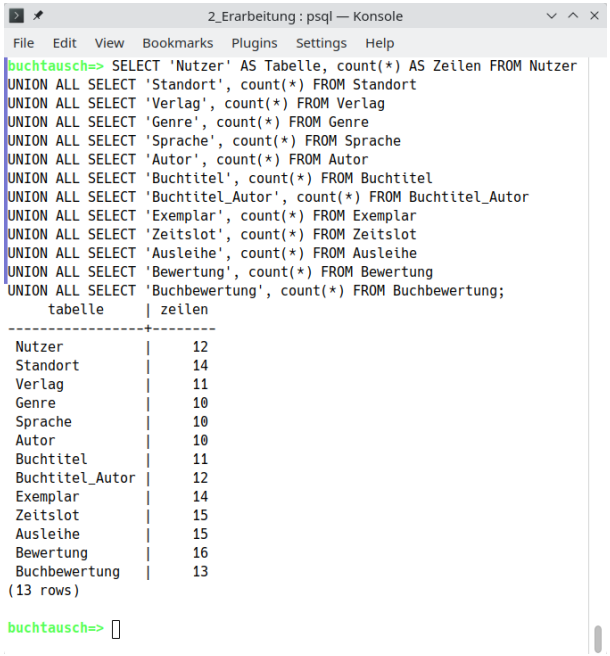


Abbildung 1: Zeilenzahl je Tabelle (psql). Quelle: Eigene Darstellung

2 Nutzer (1/12)

Diese Tabelle führt die Stammdaten der Teilnehmer. Die *EMail* ist eindeutig und dient als Anmeldename, die Telefonnummer ist optional. Ein Nutzer mit Standort, Exemplar, Ausleihe oder Bewertung lässt sich nicht löschen, da die Fremdschlüssel dieser Tabellen auf *Nutzer* restriktiv sind. Er wird stattdessen anonymisiert, der Test setzt dazu Name, Vorname, E-Mail-Adresse und Telefonnummer neu.

SQL-Dokumentation

```
-- "Nutzer" sind die Stammdaten der Teilnehmer
CREATE TABLE Nutzer (
    NutzerID      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Name          VARCHAR(100) NOT NULL,
    Vorname       VARCHAR(100) NOT NULL,
    EMail         VARCHAR(255) NOT NULL UNIQUE,
    Telefonnummer VARCHAR(30)
);
```

Testprotokoll

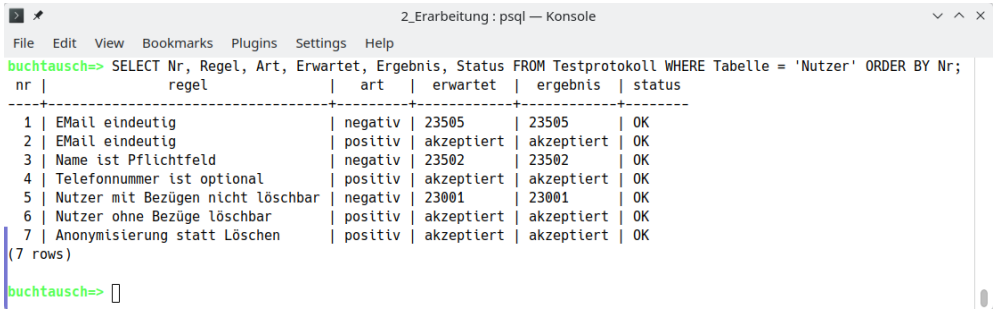


Abbildung 2: Testprotokoll der Entität Nutzer (psql). Quelle: Eigene Darstellung

Testfälle (3 negativ, 4 positiv)

```
-- Test 1, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Nutzer (Name, Vorname, EMail) VALUES ('Test', 'Neu',
   ↪  'anna.musterfrau@example.org')

-- Test 2, positiv: wird angenommen
INSERT INTO Nutzer (Name, Vorname, EMail) VALUES ('Test', 'Neu',
   ↪  'neu.test@example.org')

-- Test 3, negativ: abgelehnt mit 23502 (NOT NULL)
INSERT INTO Nutzer (Name, Vorname, EMail) VALUES (NULL, 'Neu',
   ↪  'neu.test@example.org')

-- Test 4, positiv: wird angenommen
INSERT INTO Nutzer (Name, Vorname, EMail) VALUES ('Test', 'Neu',
   ↪  'neu.test@example.org')

-- Test 5, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Nutzer WHERE NutzerID = 5

-- Test 6, positiv: wird angenommen
INSERT INTO Nutzer (Name, Vorname, EMail) VALUES ('Test', 'Löschen',
   ↪  'loeschen@example.org');
  DELETE FROM Nutzer WHERE EMail = 'loeschen@example.org'

-- Test 7, positiv: wird angenommen
UPDATE Nutzer SET Name = 'anonym', Vorname = 'anonym', EMail =
   ↪  'anonym-5@example.invalid',
        Telefonnummer = NULL WHERE NutzerID = 5
```

3 Standort (2/12)

Die Adresse eines Nutzers mit Geokoordinaten für die Umkreissuche. Breiten- und Längengrad sind auf den gültigen Wertebereich beschränkt. *UNIQUE (NutzerID, StandortID)* ist das Ziel des zusammengesetzten Fremdschlüssels aus *Exemplar*. Er erzwingt, dass der Abholort ein Standort des Besitzers ist. Ein Standort mit Exemplaren lässt sich jedoch nicht löschen. Der Index auf Breiten- und Längengrad dient der Vorauswahl bei der Umkreissuche.

SQL-Dokumentation

```
-- "Standort" als Adresse mit Koordinaten (für Umkreissuche benötigt)
CREATE TABLE Standort (
    StandortID    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    NutzerID      INTEGER NOT NULL REFERENCES Nutzer (NutzerID) ON DELETE RESTRICT,
    Straße        VARCHAR(100) NOT NULL,
    Hausnummer    VARCHAR(10)  NOT NULL,
    Postleitzahl  VARCHAR(10)  NOT NULL,
    Ort           VARCHAR(100) NOT NULL,
    Land          VARCHAR(100) NOT NULL,
    Breitengrad   NUMERIC(8,6) NOT NULL CHECK (Breitengrad BETWEEN -90 AND 90),
    Längengrad    NUMERIC(9,6) NOT NULL CHECK (Längengrad BETWEEN -180 AND 180),
    UNIQUE (NutzerID, StandortID)
);

-- Vorauswahl der Umkreissuche
CREATE INDEX idx_standort_koordinaten    ON Standort (Breitengrad, Längengrad);
```

Testprotokoll

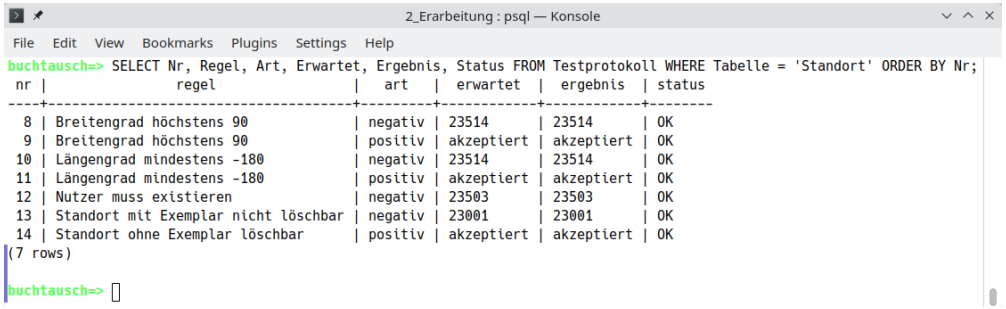


Abbildung 3: Testprotokoll der Entität Standort (psql). Quelle: Eigene Darstellung

Testfälle (4 negativ, 3 positiv)

```
-- Test 8, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Standort (NutzerID, Straße, Hausnummer, Postleitzahl, Ort, Land,
    ↪  Breitengrad, Längengrad)
  VALUES (1, 'Testweg', '1', '10115', 'Berlin', 'Deutschland', 90.500000,
      ↪  13.400000)

-- Test 9, positiv: wird angenommen
INSERT INTO Standort (NutzerID, Straße, Hausnummer, Postleitzahl, Ort, Land,
    ↪  Breitengrad, Längengrad)
  VALUES (1, 'Testweg', '1', '10115', 'Berlin', 'Deutschland', 90.000000,
      ↪  13.400000)

-- Test 10, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Standort (NutzerID, Straße, Hausnummer, Postleitzahl, Ort, Land,
    ↪  Breitengrad, Längengrad)
  VALUES (1, 'Testweg', '1', '10115', 'Berlin', 'Deutschland', 52.500000,
      ↪  -180.500000)

-- Test 11, positiv: wird angenommen
INSERT INTO Standort (NutzerID, Straße, Hausnummer, Postleitzahl, Ort, Land,
    ↪  Breitengrad, Längengrad)
  VALUES (1, 'Testweg', '1', '10115', 'Berlin', 'Deutschland', 52.500000,
      ↪  -180.000000)

-- Test 12, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Standort (NutzerID, Straße, Hausnummer, Postleitzahl, Ort, Land,
    ↪  Breitengrad, Längengrad)
  VALUES (999, 'Testweg', '1', '10115', 'Berlin', 'Deutschland', 52.500000,
      ↪  13.400000)

-- Test 13, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Standort WHERE StandortID = 1

-- Test 14, positiv: wird angenommen
INSERT INTO Standort (NutzerID, Straße, Hausnummer, Postleitzahl, Ort, Land,
    ↪  Breitengrad, Längengrad)
  VALUES (1, 'Testweg', '1', '10115', 'Berlin', 'Deutschland', 52.500000,
      ↪  13.400000);
  DELETE FROM Standort WHERE Straße = 'Testweg'
```

4 Verlag (3/12)

Die Nachschlagetabelle der Verlage. Der Name ist eindeutig, Ort und Land sind optional. Ein Verlag, auf den ein Buchtitel verweist, lässt sich nicht löschen.

SQL-Dokumentation

```
-- Verlage der Buchtitel
CREATE TABLE Verlag (
    VerlagID INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Name     VARCHAR(150) NOT NULL UNIQUE,
    Ort      VARCHAR(100),
    Land     VARCHAR(100)
);
```

Testprotokoll

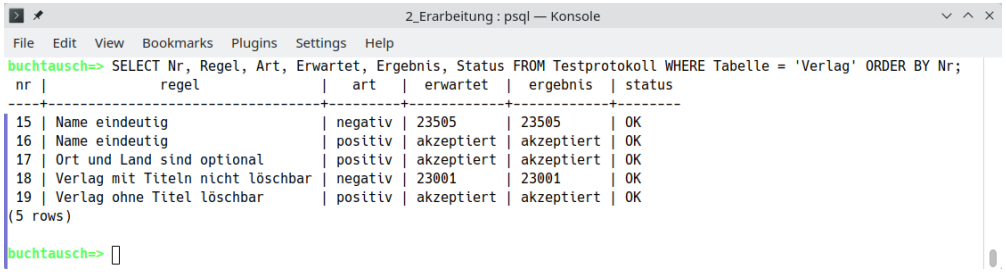


Abbildung 4: Testprotokoll der Entität Verlag (psql). Quelle: Eigene Darstellung

Testfälle (2 negativ, 3 positiv)

```
-- Test 15, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Verlag (Name) VALUES ('Musterverlag')

-- Test 16, positiv: wird angenommen
INSERT INTO Verlag (Name) VALUES ('Neuer Testverlag')

-- Test 17, positiv: wird angenommen
INSERT INTO Verlag (Name, Ort, Land) VALUES ('Neuer Testverlag', NULL, NULL)

-- Test 18, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Verlag WHERE Name = 'Musterverlag'

-- Test 19, positiv: wird angenommen
DELETE FROM Verlag WHERE Name = 'Datenverlag'
```

5 Genre (4/12)

Die Nachschlagetabelle der Genres mit eindeutiger Bezeichnung. Ein Genre, das ein Buchtitel verwendet, lässt sich nicht löschen.

SQL-Dokumentation

```
-- Genres
CREATE TABLE Genre (
    GenreID     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Bezeichnung VARCHAR(50) NOT NULL UNIQUE
);
```

Testprotokoll

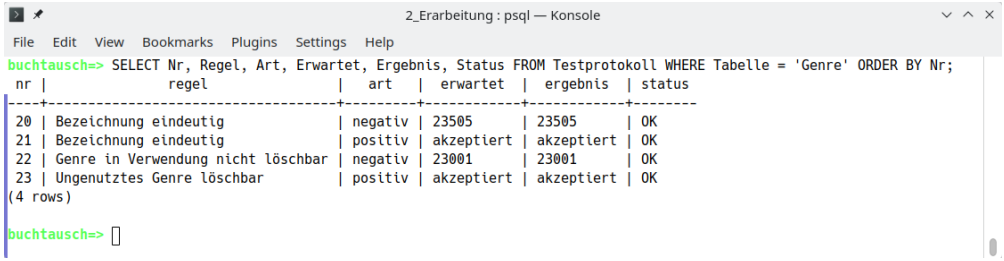


Abbildung 5: Testprotokoll der Entität Genre (psql). Quelle: Eigene Darstellung

Testfälle (2 negativ, 2 positiv)

```
-- Test 20, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Genre (Bezeichnung) VALUES ('Roman')

-- Test 21, positiv: wird angenommen
INSERT INTO Genre (Bezeichnung) VALUES ('Märchen')

-- Test 22, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Genre WHERE Bezeichnung = 'Roman'

-- Test 23, positiv: wird angenommen
DELETE FROM Genre WHERE Bezeichnung = 'Biografie'
```

6 Sprache (5/12)

Die Nachschlagetabelle der Sprachen mit eindeutiger Bezeichnung. Eine Sprache, die ein Buchtitel verwendet, lässt sich nicht löschen.

SQL-Dokumentation

```
-- Sprachen
CREATE TABLE Sprache (
    SpracheID   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Bezeichnung VARCHAR(50) NOT NULL UNIQUE
);
```

Testprotokoll

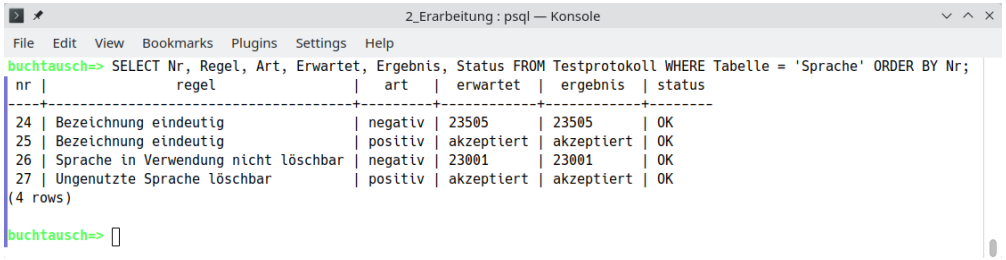


Abbildung 6: Testprotokoll der Entität Sprache (psql). Quelle: Eigene Darstellung

Testfälle (2 negativ, 2 positiv)

```
-- Test 24, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Sprache (Bezeichnung) VALUES ('Deutsch')

-- Test 25, positiv: wird angenommen
INSERT INTO Sprache (Bezeichnung) VALUES ('Dänisch')

-- Test 26, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Sprache WHERE Bezeichnung = 'Deutsch'

-- Test 27, positiv: wird angenommen
DELETE FROM Sprache WHERE Bezeichnung = 'Portugiesisch'
```

7 Autor (6/12)

Die Autoren der Buchtitel. Der Name ist Pflicht, der Vorname ist optional. Ein Autor, dem ein Buchtitel zugeordnet ist, lässt sich nicht löschen.

SQL-Dokumentation

```
-- Autoren der Buchtitel
CREATE TABLE Autor (
    AutorID INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Name    VARCHAR(100) NOT NULL,
    Vorname VARCHAR(100)
);
```

Testprotokoll

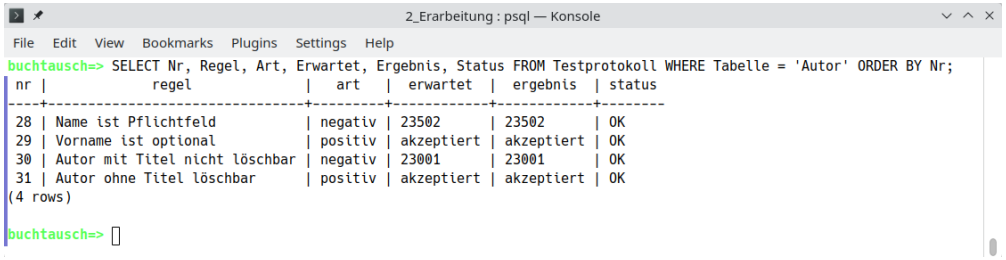


Abbildung 7: Testprotokoll der Entität Autor (psql). Quelle: Eigene Darstellung

Testfälle (2 negativ, 2 positiv)

```
-- Test 28, negativ: abgelehnt mit 23502 (NOT NULL)
INSERT INTO Autor (Name, Vorname) VALUES (NULL, 'Test')

-- Test 29, positiv: wird angenommen
INSERT INTO Autor (Name, Vorname) VALUES ('Pseudonym', NULL)

-- Test 30, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Autor WHERE Name = 'Mustermann'

-- Test 31, positiv: wird angenommen
INSERT INTO Autor (Name, Vorname) VALUES ('Pseudonym', NULL);
  DELETE FROM Autor WHERE Name = 'Pseudonym'
```

8 Buchtitel (7/12)

Die Bibliografische Angaben, unabhängig vom einzelnen Exemplar. Die ISBN ist optional und eindeutig, Erscheinungsjahr und Auflage sind durch *CHECK* begrenzt, der Verlag ist optional. Genre und Sprache müssen existieren. Mit dem Titel entfällt die Zuordnung des Autors, ein Titel mit Exemplar lässt sich aber nicht löschen. Die Zuordnungstabelle *Buchtitel_Autor* ist keine Entität, daher hat sie keine eigene Folie. Sie löst die Beziehung zu *Autor* auf. Der zusammengesetzte Primärschlüssel verhindert doppelte Zuordnungen, also kann ein Titel kann mehrere Autoren haben. Die Zuordnung entfällt mit dem Titel (*CASCADE*), ein Autor mit Titeln bleibt dagegen geschützt (*RESTRICT*).

SQL-Dokumentation

```
-- Buchtitel also bibliografische Angaben
CREATE TABLE Buchtitel (
    BuchtitelID      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    VerlagID         INTEGER REFERENCES Verlag (VerlagID) ON DELETE RESTRICT,
    GenreID          INTEGER NOT NULL REFERENCES Genre (GenreID) ON DELETE RESTRICT,
    SpracheID        INTEGER NOT NULL REFERENCES Sprache (SpracheID) ON DELETE RESTRICT,
    ISBN             VARCHAR(17) UNIQUE,
    Titel            VARCHAR(255) NOT NULL,
    Erscheinungsjahr SMALLINT NOT NULL CHECK (Erscheinungsjahr BETWEEN 1450 AND 2100),
    Auflage          SMALLINT NOT NULL DEFAULT 1 CHECK (Auflage > 0)
);

-- Zuordnungstabelle von Buchtitel zu Autor
CREATE TABLE Buchtitel_Autor (
    BuchtitelID INTEGER NOT NULL REFERENCES Buchtitel (BuchtitelID) ON DELETE CASCADE,
    AutorID     INTEGER NOT NULL REFERENCES Autor (AutorID) ON DELETE RESTRICT,
    PRIMARY KEY (BuchtitelID, AutorID)
);
```

Testfälle (7 negativ, 6 positiv)

```
-- Test 32, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Buchtitel (GenreID, SpracheID, ISBN, Titel, Erscheinungsjahr)
  VALUES (1, 1, '978-3-99999-001-9', 'Doppelte ISBN', 2020)

-- Test 33, positiv: wird angenommen
INSERT INTO Buchtitel (GenreID, SpracheID, ISBN, Titel, Erscheinungsjahr)
  VALUES (1, 1, NULL, 'Titel ohne ISBN', 2020)

-- Test 34, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Buchtitel (GenreID, SpracheID, Titel, Erscheinungsjahr)
  VALUES (1, 1, 'Zu alt', 1449)

-- Test 35, positiv: wird angenommen
INSERT INTO Buchtitel (GenreID, SpracheID, Titel, Erscheinungsjahr)
  VALUES (1, 1, 'Frühdruck', 1450)

-- Test 36, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Buchtitel (GenreID, SpracheID, Titel, Erscheinungsjahr)
  VALUES (1, 1, 'Zu neu', 2101)

-- Test 37, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Buchtitel (GenreID, SpracheID, Titel, Erscheinungsjahr, Auflage)
  VALUES (1, 1, 'Nullte Auflage', 2020, 0)

-- Test 38, positiv: wird angenommen
INSERT INTO Buchtitel (GenreID, SpracheID, Titel, Erscheinungsjahr)
  VALUES (1, 1, 'Standardauflage', 2020)

-- Test 39, positiv: wird angenommen
INSERT INTO Buchtitel (VerlagID, GenreID, SpracheID, Titel, Erscheinungsjahr)
  VALUES (NULL, 1, 1, 'Titel ohne Verlag', 2020)

-- Test 40, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Buchtitel (GenreID, SpracheID, Titel, Erscheinungsjahr)
  VALUES (999, 1, 'Unbekanntes Genre', 2020)

-- Test 41, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Buchtitel WHERE BuchtitelID = 1

-- Test 43, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Buchtitel_Autor (BuchtitelID, AutorID) VALUES (1, 1)
```

Buchtitel, Fortsetzung

Testprotokoll

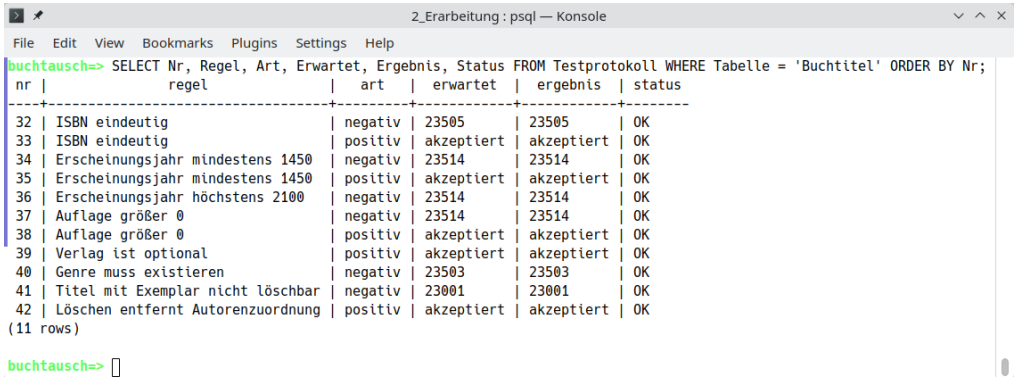


Abbildung 8: Testprotokoll der Entität Buchtitel (psql). Quelle: Eigene Darstellung

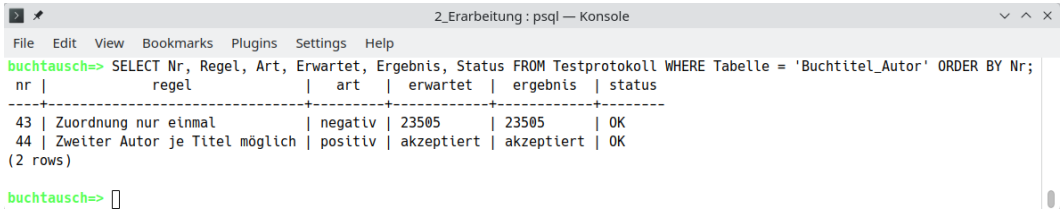


Abbildung 9: Testprotokoll der Zuordnungstabelle Buchtitel_Autor (psql). Quelle: Eigene Darstellung

```
-- Test 42, positiv: wird angenommen
DO $$ BEGIN
      DELETE FROM Buchtitel WHERE BuchtitelID = 11;
      IF EXISTS (SELECT 1 FROM Buchtitel_Autor WHERE BuchtitelID = 11) THEN
          RAISE EXCEPTION 'Zuordnung blieb bestehen';
      END IF;
  END $$

-- Test 44, positiv: wird angenommen
INSERT INTO Buchtitel_Autor (BuchtitelID, AutorID) VALUES (1, 2)
```

9 Exemplar (8/12)

Das tatsächliche, physische Buch eines Besitzers. Der zusammengesetzte Fremdschlüssel *(BesitzerNutzerID, StandortID)* erzwingt, dass der Abholort dem Besitzer gehört. Die möglichen Zustände sind auf Neu, Gut oder Gebraucht beschränkt, die maximale Ausleihdauer muss positiv sein, der Versand ist standardmäßig nicht möglich. Ein Exemplar mit Ausleihe lässt sich nicht löschen.

SQL-Dokumentation

```
-- Exemplar, also ein physisches Buch eines Besitzers mit Abholort
CREATE TABLE Exemplar (
    ExemplarID          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    BuchtitelID         INTEGER NOT NULL REFERENCES Buchtitel (BuchtitelID) ON DELETE RESTRICT,
    BesitzerNutzerID    INTEGER NOT NULL REFERENCES Nutzer (NutzerID) ON DELETE RESTRICT,
    StandortID          INTEGER NOT NULL,
    Zustand             VARCHAR(20) NOT NULL CHECK (Zustand IN ('Neu', 'Gut', 'Gebraucht')),
    MaxAusleihDauerTage SMALLINT NOT NULL CHECK (MaxAusleihDauerTage > 0),
    VersandMöglich      BOOLEAN NOT NULL DEFAULT FALSE,
    FOREIGN KEY (BesitzerNutzerID, StandortID)
        REFERENCES Standort (NutzerID, StandortID) ON DELETE RESTRICT
);

-- Join von Exemplar zu Buchtitel
CREATE INDEX idx_exemplar_buchtitel      ON Exemplar (BuchtitelID);
```

Testprotokoll

2_Erarbeitung : psql — Konsole

File Edit View Bookmarks Plugins Settings Help

```
buchtausch=> SELECT Nr, Regel, Art, Erwartet, Ergebnis, Status FROM Testprotokoll WHERE Tabelle = 'Exemplar' ORDER BY Nr;
 nr |                regel                |   art    |  erwartet  |  ergebnis  | status
----+-------------------------------------+----------+------------+------------+--------
 45 | Zustand Neu, Gut oder Gebraucht     | negativ  | 23514      | 23514      | OK
 46 | Zustand Neu, Gut oder Gebraucht     | positiv  | akzeptiert | akzeptiert | OK
 47 | Maximale Ausleihdauer größer 0      | negativ  | 23514      | 23514      | OK
 48 | Maximale Ausleihdauer größer 0      | positiv  | akzeptiert | akzeptiert | OK
 49 | Abholort gehört dem Besitzer        | negativ  | 23503      | 23503      | OK
 50 | Abholort gehört dem Besitzer        | positiv  | akzeptiert | akzeptiert | OK
 51 | Versand standardmäßig nicht möglich | positiv  | akzeptiert | akzeptiert | OK
 63 | Exemplar mit Ausleihe nicht löschbar | negativ | 23001      | 23001      | OK
(8 rows)

buchtausch=>
```

Abbildung 10: Testprotokoll der Entität Exemplar (psql). Quelle: Eigene Darstellung

Testfälle (4 negativ, 4 positiv)

```
-- Test 45, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Exemplar (BuchtitelID, BesitzerNutzerID, StandortID, Zustand,
    ↪  MaxAusleihDauerTage)
  VALUES (1, 1, 1, 'Defekt', 14)

-- Test 46, positiv: wird angenommen
INSERT INTO Exemplar (BuchtitelID, BesitzerNutzerID, StandortID, Zustand,
    ↪  MaxAusleihDauerTage)
  VALUES (1, 1, 1, 'Neu', 14)

-- Test 47, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Exemplar (BuchtitelID, BesitzerNutzerID, StandortID, Zustand,
    ↪  MaxAusleihDauerTage)
  VALUES (1, 1, 1, 'Gut', 0)

-- Test 48, positiv: wird angenommen
INSERT INTO Exemplar (BuchtitelID, BesitzerNutzerID, StandortID, Zustand,
    ↪  MaxAusleihDauerTage)
  VALUES (1, 1, 1, 'Gut', 1)

-- Test 49, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Exemplar (BuchtitelID, BesitzerNutzerID, StandortID, Zustand,
    ↪  MaxAusleihDauerTage)
  VALUES (1, 1, 2, 'Gut', 14)

-- Test 50, positiv: wird angenommen
INSERT INTO Exemplar (BuchtitelID, BesitzerNutzerID, StandortID, Zustand,
    ↪  MaxAusleihDauerTage)
  VALUES (1, 1, 13, 'Gut', 14)

-- Test 51, positiv: wird angenommen
DO $$ DECLARE v BOOLEAN; BEGIN
      INSERT INTO Exemplar (BuchtitelID, BesitzerNutzerID, StandortID, Zustand,
         ↪  MaxAusleihDauerTage)
      VALUES (1, 1, 1, 'Gut', 14) RETURNING VersandMöglich INTO v;
      IF v IS DISTINCT FROM FALSE THEN RAISE EXCEPTION 'Standardwert ist nicht
         ↪  FALSE'; END IF;
  END $$

-- Test 63, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Exemplar WHERE ExemplarID = 1
```

10 Zeitslot (9/12)

Ein täglich wiederkehrendes Abholfenster eines Exemplars innerhalb eines Zeitraums. *CHECK*-Bedingungen sichern die Reihenfolge von Tagen und Uhrzeiten. Der *EXCLUDE*-Constraint verhindert, dass sich zwei Slots desselben Exemplars in Datum und Uhrzeit überlappen. Mit dem Exemplar entfallen auch seine Slots, ein Slot mit Ausleihe bleibt dagegen bestehen.

SQL-Dokumentation

```
-- Zeitslot ist das Abholfenster eines Exemplars
CREATE TABLE Zeitslot (
    ZeitslotID  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ExemplarID  INTEGER NOT NULL REFERENCES Exemplar (ExemplarID) ON DELETE CASCADE,
    Tag_von     DATE NOT NULL,
    Tag_bis     DATE NOT NULL,
    Uhrzeit_von TIME NOT NULL,
    Uhrzeit_bis TIME NOT NULL,
    CHECK (Tag_bis >= Tag_von),
    CHECK (Uhrzeit_bis > Uhrzeit_von),
    UNIQUE (ExemplarID, ZeitslotID),
    EXCLUDE USING gist (
        ExemplarID WITH =,
        daterange(Tag_von, Tag_bis, '[]') WITH &&,
        timerange(Uhrzeit_von, Uhrzeit_bis) WITH &&
    )
);
```

Testprotokoll

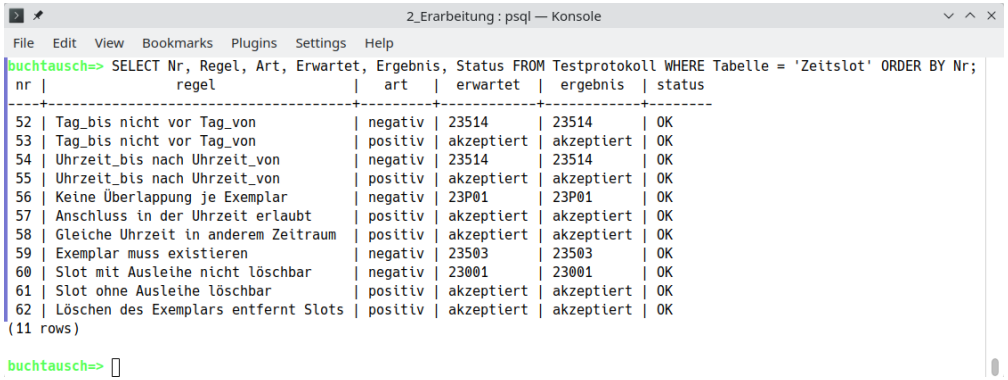


Abbildung 11: Testprotokoll der Entität Zeitslot (psql). Quelle: Eigene Darstellung

Testfälle (5 negativ, 6 positiv)

```
-- Test 52, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (3, '2026-08-10', '2026-08-01', '18:00', '20:00')

-- Test 53, positiv: wird angenommen
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (3, '2026-08-10', '2026-08-10', '18:00', '20:00')

-- Test 54, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (3, '2026-08-10', '2026-08-10', '20:00', '18:00')

-- Test 55, positiv: wird angenommen
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (3, '2026-08-10', '2026-08-10', '18:00', '18:01')

-- Test 56, negativ: abgelehnt mit 23P01 (EXCLUDE)
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (1, '2026-06-01', '2026-06-30', '18:00', '20:00')

-- Test 57, positiv: wird angenommen
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (2, '2026-01-01', '2026-12-31', '18:00', '20:00')

-- Test 58, positiv: wird angenommen
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (3, '2026-07-01', '2026-07-31', '18:00', '20:00')

-- Test 59, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Zeitslot (ExemplarID, Tag_von, Tag_bis, Uhrzeit_von, Uhrzeit_bis)
  VALUES (999, '2026-01-01', '2026-01-31', '18:00', '20:00')

-- Test 60, negativ: abgelehnt mit 23001 (RESTRICT)
DELETE FROM Zeitslot WHERE ZeitslotID = 1

-- Test 61, positiv: wird angenommen
DELETE FROM Zeitslot WHERE ZeitslotID = 12

-- Test 62, positiv: wird angenommen
DO $$ BEGIN
      DELETE FROM Exemplar WHERE ExemplarID = 12;
      IF EXISTS (SELECT 1 FROM Zeitslot WHERE ExemplarID = 12) THEN
          RAISE EXCEPTION 'Slots blieben bestehen';
      END IF;
  END $$
```

11 Ausleihe (10/12)

Verknüpft Exemplar, Zeitslot, Ausleiher und Verleiher miteinander. Der zusammengesetzte Fremdschlüssel erzwingt, dass der Slot zum Exemplar gehört. Der *EXCLUDE*-Constraint verhindert überlappende Verleihzeiträume und erlaubt die Übergabe am Rückgabetag. Zwei Regeln prüft der Trigger *Ausleihe_Regeln* (Abschnitt Trigger): Der Verleiher ist der Besitzer des Exemplars und *RückgabeSpätestens* ergibt sich aus Ausleihdatum und *MaxAusleihDauerTage*.

SQL-Dokumentation

```
-- Ausleihe verknüpft Exemplar, Zeitslot, Ausleiher und Verleiher
CREATE TABLE Ausleihe (
    AusleiheID         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ExemplarID         INTEGER NOT NULL REFERENCES Exemplar (ExemplarID) ON DELETE RESTRICT,
    ZeitslotID         INTEGER NOT NULL,
    AusleiherNutzerID  INTEGER NOT NULL REFERENCES Nutzer (NutzerID) ON DELETE RESTRICT,
    VerleiherNutzerID  INTEGER NOT NULL REFERENCES Nutzer (NutzerID) ON DELETE RESTRICT,
    Ausleihdatum       DATE NOT NULL DEFAULT CURRENT_DATE,
    RückgabeSpätestens DATE NOT NULL,
    Rückgabedatum      DATE,
    CHECK (VerleiherNutzerID <> AusleiherNutzerID),
    CHECK (RückgabeSpätestens > Ausleihdatum),
    CHECK (Rückgabedatum >= Ausleihdatum),
    FOREIGN KEY (ExemplarID, ZeitslotID)
        REFERENCES Zeitslot (ExemplarID, ZeitslotID) ON DELETE RESTRICT,
    EXCLUDE USING gist (
        ExemplarID WITH =,
        daterange(Ausleihdatum, RückgabeSpätestens, '[)') WITH &&
    )
);

-- Join von Ausleihe zu Exemplar und Zeitslot
CREATE INDEX idx_ausleihe_exemplar_slot  ON Ausleihe (ExemplarID, ZeitslotID);
-- Join von Ausleihe zum Ausleiher
CREATE INDEX idx_ausleihe_ausleiher      ON Ausleihe (AusleiherNutzerID);
```

Testfälle (7 negativ, 5 positiv)

```
-- Test 64, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID,
   ↪  VerleiherNutzerID, Ausleihdatum)
  VALUES (1, 1, 1, 1, '2026-09-01')

-- Test 65, positiv: wird angenommen
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID,
   ↪  VerleiherNutzerID, Ausleihdatum)
  VALUES (2, 2, 4, 2, '2026-11-02')

-- Test 66, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID,
   ↪  VerleiherNutzerID, Ausleihdatum)
  VALUES (3, 3, 5, 4, '2026-09-01')

-- Test 67, positiv: wird angenommen
UPDATE Exemplar SET BesitzerNutzerID = 2, StandortID = 2 WHERE ExemplarID = 1;
  UPDATE Ausleihe SET Rückgabedatum = '2026-02-16' WHERE AusleiheID = 1

-- Test 68, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID,
   ↪  VerleiherNutzerID, Ausleihdatum, RückgabeSpätestens)
  VALUES (2, 2, 4, 2, '2026-11-02', '2026-11-30')

-- Test 69, positiv: wird angenommen
DO $$ DECLARE v DATE; BEGIN
      INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID,
         ↪  VerleiherNutzerID, Ausleihdatum)
      VALUES (2, 2, 4, 2, '2026-11-02') RETURNING RückgabeSpätestens INTO v;
      IF v <> DATE '2026-11-23' THEN RAISE EXCEPTION 'Falsche Frist %', v; END
         ↪  IF;
  END $$

-- Test 70, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID,
   ↪  VerleiherNutzerID, Ausleihdatum, Rückgabedatum)
  VALUES (2, 2, 4, 2, '2026-11-02', '2026-11-01')

-- Test 71, positiv: wird angenommen
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID,
   ↪  VerleiherNutzerID, Ausleihdatum, Rückgabedatum)
  VALUES (2, 2, 4, 2, '2026-11-02', '2026-11-02')
```

Ausleihe, Fortsetzung

Testprotokoll

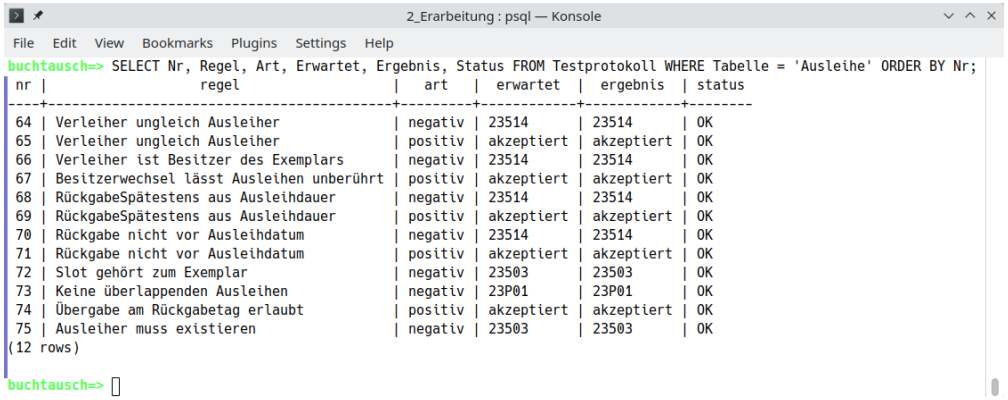


Abbildung 12: Testprotokoll der Entität Ausleihe (psql). Quelle: Eigene Darstellung

```
-- Test 72, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID, VerleiherNutzerID, Ausleihdatum)
  VALUES (2, 1, 4, 2, '2026-11-02')

-- Test 73, negativ: abgelehnt mit 23P01 (EXCLUDE)
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID, VerleiherNutzerID, Ausleihdatum)
  VALUES (1, 1, 3, 1, '2026-03-10')

-- Test 74, positiv: wird angenommen
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID, VerleiherNutzerID, Ausleihdatum)
  VALUES (1, 1, 3, 1, '2026-03-16')

-- Test 75, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Ausleihe (ExemplarID, ZeitslotID, AusleiherNutzerID, VerleiherNutzerID, Ausleihdatum)
  VALUES (2, 2, 999, 2, '2026-11-02')
```

12 Bewertung (11/12)

Die gegenseitige Bewertung der Beteiligten einer Ausleihe mit einem bis fünf Sternen. Je Ausleihe und Bewertendem ist höchstens eine Bewertung zulässig. Der Trigger *Bewertung_Regeln* (Abschnitt Trigger) stellt sicher, dass Bewertender und Bewerteter Ausleiher und Verleiher der Ausleihe sind. Anwendungsregel ohne Prüfung in der Datenbank: Bewertet wird erst nach der Rückgabe.

SQL-Dokumentation

```
-- Bewertung einer Ausleihe durch alle Beteiligten
CREATE TABLE Bewertung (
    BewertungID         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    AusleiheID          INTEGER NOT NULL REFERENCES Ausleihe (AusleiheID) ON DELETE RESTRICT,
    BewertenderNutzerID INTEGER NOT NULL REFERENCES Nutzer (NutzerID) ON DELETE RESTRICT,
    BewerteterNutzerID  INTEGER NOT NULL REFERENCES Nutzer (NutzerID) ON DELETE RESTRICT,
    Sterne              SMALLINT NOT NULL CHECK (Sterne BETWEEN 1 AND 5),
    Kommentar           TEXT,
    Datum               DATE NOT NULL DEFAULT CURRENT_DATE,
    CHECK (BewerteterNutzerID <> BewertenderNutzerID),
    UNIQUE (AusleiheID, BewertenderNutzerID)
);

-- Join von Bewertung zum bewerteten Nutzer
CREATE INDEX idx_bewertung_bewerteter    ON Bewertung (BewerteterNutzerID);
```

Testprotokoll

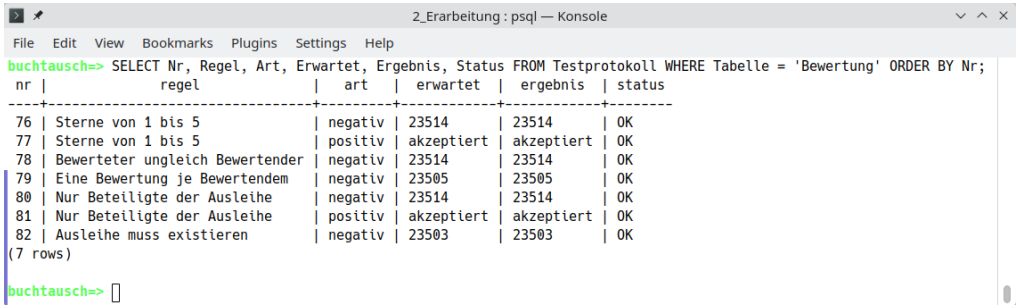


Abbildung 13: Testprotokoll der Entität Bewertung (psql). Quelle: Eigene Darstellung

Testfälle (5 negativ, 2 positiv)

```
-- Test 76, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Bewertung (AusleiheID, BewertenderNutzerID, BewerteterNutzerID,
    ↪  Sterne)
  VALUES (7, 6, 11, 6)

-- Test 77, positiv: wird angenommen
INSERT INTO Bewertung (AusleiheID, BewertenderNutzerID, BewerteterNutzerID,
    ↪  Sterne)
  VALUES (7, 6, 11, 1)

-- Test 78, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Bewertung (AusleiheID, BewertenderNutzerID, BewerteterNutzerID,
    ↪  Sterne)
  VALUES (7, 6, 6, 3)

-- Test 79, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Bewertung (AusleiheID, BewertenderNutzerID, BewerteterNutzerID,
    ↪  Sterne)
  VALUES (1, 5, 1, 3)

-- Test 80, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Bewertung (AusleiheID, BewertenderNutzerID, BewerteterNutzerID,
    ↪  Sterne)
  VALUES (1, 3, 1, 4)

-- Test 81, positiv: wird angenommen
INSERT INTO Bewertung (AusleiheID, BewertenderNutzerID, BewerteterNutzerID,
    ↪  Sterne)
  VALUES (15, 2, 9, 5)

-- Test 82, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Bewertung (AusleiheID, BewertenderNutzerID, BewerteterNutzerID,
    ↪  Sterne)
  VALUES (999, 5, 1, 4)
```

13 Buchbewertung (12/12)

Die Bewertung eines Buchtitels durch einen Nutzer mit einem bis fünf Sternen. Pro Nutzer und Titel ist höchstens eine Bewertung zulässig - Titel und Nutzer müssen existieren.

SQL-Dokumentation

```
-- Bewertung eines Buchtitels durch einen Nutzer
CREATE TABLE Buchbewertung (
    BuchbewertungID INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    BuchtitelID     INTEGER NOT NULL REFERENCES Buchtitel (BuchtitelID) ON DELETE RESTRICT,
    NutzerID        INTEGER NOT NULL REFERENCES Nutzer (NutzerID) ON DELETE RESTRICT,
    Sterne          SMALLINT NOT NULL CHECK (Sterne BETWEEN 1 AND 5),
    Kommentar       TEXT,
    Datum           DATE NOT NULL DEFAULT CURRENT_DATE,
    UNIQUE (BuchtitelID, NutzerID)
);
```

Testprotokoll

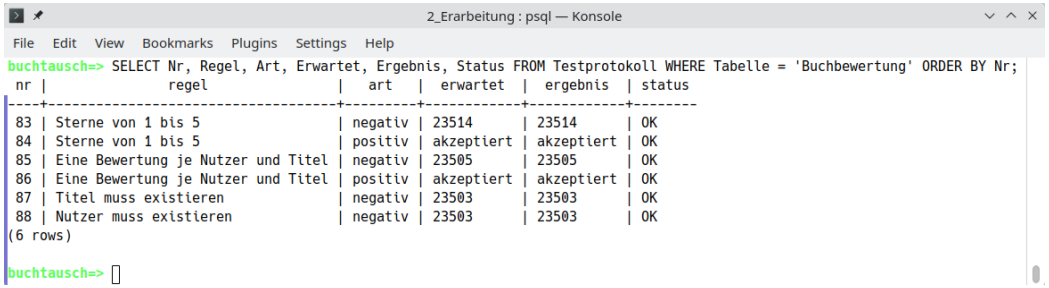


Abbildung 14: Testprotokoll der Entität Buchbewertung (psql). Quelle: Eigene Darstellung

Testfälle (4 negativ, 2 positiv)

```
-- Test 83, negativ: abgelehnt mit 23514 (CHECK)
INSERT INTO Buchbewertung (BuchtitelID, NutzerID, Sterne) VALUES (4, 1, 0)

-- Test 84, positiv: wird angenommen
INSERT INTO Buchbewertung (BuchtitelID, NutzerID, Sterne) VALUES (4, 1, 5)

-- Test 85, negativ: abgelehnt mit 23505 (UNIQUE)
INSERT INTO Buchbewertung (BuchtitelID, NutzerID, Sterne) VALUES (1, 5, 4)

-- Test 86, positiv: wird angenommen
INSERT INTO Buchbewertung (BuchtitelID, NutzerID, Sterne) VALUES (1, 7, 4)

-- Test 87, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Buchbewertung (BuchtitelID, NutzerID, Sterne) VALUES (999, 1, 3)

-- Test 88, negativ: abgelehnt mit 23503 (Fremdschlüssel)
INSERT INTO Buchbewertung (BuchtitelID, NutzerID, Sterne) VALUES (1, 999, 3)
```

14 Trigger

Die Konzeption speichert *VerleiherNutzerID*, *RückgabeSpätestens* und *BewerteterNutzerID* bewusst, obwohl sie auch ableitbar wären. Drei Regeln sichern, dass die Werte zum Ableitbaren passen. Sie lassen sich nicht als Bedingung einer einzelnen Zeile formulieren, weil sie Werte einer anderen Tabelle heranziehen. Sie sind als *BEFORE*-Trigger umgesetzt und melden eine Verletzung mit dem SQLSTATE 23514 (*check_violation*), sodass sie sich in den Tests wie eine *CHECK*-Bedingung verhalten. Der Verleiher ist bewusst nicht über einen Fremdschlüssel an den Besitzer gebunden, sonst würde ein späterer Besitzerwechsel abgeschlossene Ausleihen blockieren. Die Prüfungen laufen deshalb nur beim Einfügen und bei Änderung der beteiligten Spalten.

Ausleihe: Verleiher ist Besitzer, RückgabeSpätestens aus der Ausleihdauer

```
-- Ausleihe: Verleiher ist der Besitzer, RückgabeSpätestens aus der Ausleihdauer
CREATE FUNCTION Ausleihe_Regeln() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE
    v_besitzer INTEGER;
    v_max      SMALLINT;
BEGIN
    SELECT BesitzerNutzerID, MaxAusleihDauerTage INTO v_besitzer, v_max
      FROM Exemplar WHERE ExemplarID = NEW.ExemplarID;

    IF TG_OP = 'INSERT'
       OR (NEW.ExemplarID, NEW.VerleiherNutzerID)
          IS DISTINCT FROM (OLD.ExemplarID, OLD.VerleiherNutzerID) THEN
        IF NEW.VerleiherNutzerID <> v_besitzer THEN
            RAISE EXCEPTION 'Verleiher % ist nicht der Besitzer % des Exemplars %',
                NEW.VerleiherNutzerID, v_besitzer, NEW.ExemplarID
                USING ERRCODE = 'check_violation';
        END IF;
    END IF;

    IF TG_OP = 'INSERT'
       OR (NEW.ExemplarID, NEW.Ausleihdatum, NEW.RückgabeSpätestens)
          IS DISTINCT FROM (OLD.ExemplarID, OLD.Ausleihdatum, OLD.RückgabeSpätestens) THEN
        IF NEW.RückgabeSpätestens IS NULL THEN
            NEW.RückgabeSpätestens := NEW.Ausleihdatum + v_max;
        ELSIF NEW.RückgabeSpätestens <> NEW.Ausleihdatum + v_max THEN
            RAISE EXCEPTION 'RückgabeSpätestens muss % sein (Ausleihdatum + % Tage)',
                NEW.Ausleihdatum + v_max, v_max
                USING ERRCODE = 'check_violation';
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

-- Ruft die Ausleihe_Regeln vor INSERT und UPDATE auf
CREATE TRIGGER Ausleihe_Regeln_Trigger
    BEFORE INSERT OR UPDATE OF ExemplarID, VerleiherNutzerID, Ausleihdatum, RückgabeSpätestens
    ON Ausleihe
    FOR EACH ROW EXECUTE FUNCTION Ausleihe_Regeln();
```

Bewertung: Bewertender und Bewerteter sind die Beteiligten der Ausleihe

```
-- Bewertender und Bewerteter sind Ausleiher und Verleiher
CREATE FUNCTION Bewertung_Regeln() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE
    v_ausleiher INTEGER;
    v_verleiher INTEGER;
BEGIN
    SELECT AusleiherNutzerID, VerleiherNutzerID INTO v_ausleiher, v_verleiher
      FROM Ausleihe WHERE AusleiheID = NEW.AusleiheID;

    IF NOT (NEW.BewertenderNutzerID IN (v_ausleiher, v_verleiher)
        AND NEW.BewerteterNutzerID  IN (v_ausleiher, v_verleiher)) THEN
        RAISE EXCEPTION 'Bewertender % und Bewerteter % müssen Ausleiher % und
            ↪  Verleiher % der Ausleihe % sein',
            NEW.BewertenderNutzerID, NEW.BewerteterNutzerID, v_ausleiher,
                ↪  v_verleiher, NEW.AusleiheID
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$;

-- Ruft Bewertung_Regeln vor INSERT und UPDATE auf
CREATE TRIGGER Bewertung_Regeln_Trigger
    BEFORE INSERT OR UPDATE OF AusleiheID, BewertenderNutzerID,
        ↪  BewerteterNutzerID
    ON Bewertung
    FOR EACH ROW EXECUTE FUNCTION Bewertung_Regeln();
```

15 Testverfahren (Zusatz)

Zusatz zur Aufgabenstellung. Die Folie erklärt, wie die Testfälle der Entitätsfolien ausgeführt und protokolliert werden, damit die Testprotokolle nachvollziehbar sind.

Jeder Test ruft die Prozedur *pruefe* auf. Sie führt das übergebene Statement aus, hält den SQLSTATE oder die Annahme fest und nimmt die Änderung über eine Ausnahme mit dem eigenen Code RB000 wieder zurück, sodass die Daten unverändert bleiben. Ergebnis, Erwartung und Status (OK oder FEHLER) stehen in der temporären Tabelle *Testprotokoll*. Ein negativer Test erwartet den SQLSTATE der verletzten Bedingung, ein positiver Test erwartet, dass das Statement angenommen wird. Alle 88 Tests entsprechen der Erwartung, davon 47 negative und 41 positive.

SQL-Dokumentation: Testprotokoll und Prozedur

```
-- Testprotokoll und Prüfverfahren - nur in dieser Sitzung vorhanden
DROP TABLE IF EXISTS pg_temp.Testprotokoll;
CREATE TEMP TABLE Testprotokoll (
    Nr       INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Tabelle  TEXT NOT NULL,
    Regel    TEXT NOT NULL,
    Art      TEXT NOT NULL CHECK (Art IN ('positiv', 'negativ')),
    Erwartet TEXT NOT NULL,
    Ergebnis TEXT NOT NULL,
    Status   TEXT NOT NULL
);

CREATE OR REPLACE PROCEDURE pg_temp.pruefe(p_tabelle TEXT, p_regel TEXT, p_art TEXT, p_erwartet TEXT, p_sql
   ↪  TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_ergebnis TEXT;
BEGIN
    BEGIN
        EXECUTE p_sql;
        v_ergebnis := 'akzeptiert';
        RAISE EXCEPTION 'Test zurückgenommen' USING ERRCODE = 'RB000';
    EXCEPTION WHEN OTHERS THEN
        IF SQLSTATE <> 'RB000' THEN
            v_ergebnis := SQLSTATE;
        END IF;
    END;
    INSERT INTO Testprotokoll (Tabelle, Regel, Art, Erwartet, Ergebnis, Status)
    VALUES (p_tabelle, p_regel, p_art, p_erwartet, v_ergebnis,
            CASE WHEN v_ergebnis = p_erwartet THEN 'OK' ELSE 'FEHLER' END);
END;
$$;
```

Erwartete Fehlercodes (SQLSTATE)

Code	Bedeutung
23502	NOT NULL verletzt
23503	Fremdschlüssel verletzt (Einfügen mit unbekanntem Bezug)
23505	UNIQUE verletzt
23514	CHECK verletzt, auch durch die Trigger
23001	RESTRICT verletzt (Löschen eines noch referenzierten Datensatzes, ab PostgreSQL 18)
23P01	Ausschluss-Constraint verletzt (Überlappung)

Anzahl der Tests je Tabelle

Tabelle	negativ	positiv	gesamt
Nutzer	3	4	7
Standort	4	3	7
Verlag	2	3	5
Genre	2	2	4
Sprache	2	2	4
Autor	2	2	4
Buchtitel	6	5	11
Buchtitel_Autor	1	1	2
Exemplar	4	4	8
Zeitslot	5	6	11
Ausleihe	7	5	12
Bewertung	5	2	7
Buchbewertung	4	2	6
Summe	**47**	**41**	**88**

Abfrage des Testprotokolls je Entität

```
SELECT Nr, Regel, Art, Erwartet, Ergebnis, Status
  FROM Testprotokoll WHERE Tabelle = 'Nutzer' ORDER BY Nr;
```

16 Abfragen 1 und 2

Da die Aufgabenstellung einige realisierbare Abfragen voraussetzt, werden diese folgend illustriert. Diese stehen auch in der SQL-Datei.

Abfrage 1: Umkreissuche

2_Erarbeitung : psql — Konsole

File Edit View Bookmarks Plugins Settings Help

```
buchtausch=> WITH Entfernung AS (
    SELECT e.ExemplarID, e.BuchtitelID, s.Ort,
           6371 * acos(sin(radians(m.Breitengrad)) * sin(radians(s.Breitengrad))
                + cos(radians(m.Breitengrad)) * cos(radians(s.Breitengrad))
                * cos(radians(s.Längengrad - m.Längengrad))) AS km
      FROM Standort m, Exemplar e
      JOIN Standort s ON s.NutzerID = e.BesitzerNutzerID AND s.StandortID = e.StandortID
     WHERE m.NutzerID = 3 AND m.StandortID = 3
       AND s.Breitengrad BETWEEN m.Breitengrad - 600 / 111.2 AND m.Breitengrad + 600 / 111.2
)
SELECT d.ExemplarID, b.Titel, d.Ort, round(d.km::numeric, 1) AS Entfernung_km
  FROM Entfernung d
  JOIN Buchtitel b ON b.BuchtitelID = d.BuchtitelID
 WHERE b.Titel = 'Zwischen Tabelle und Zeile'
   AND d.km <= 600
   AND NOT EXISTS (SELECT 1 FROM Ausleihe a
                   WHERE a.ExemplarID = d.ExemplarID AND a.Rückgabedatum IS NULL)
 ORDER BY d.km;
 exemplarid |           titel            |   ort   | entfernung_km
------------+----------------------------+---------+---------------
         14 | Zwischen Tabelle und Zeile | Hamburg |         254.7
          5 | Zwischen Tabelle und Zeile | München |         506.7
(2 rows)

buchtausch=>
```

Abbildung 15: Ausgabe der Abfrage 1 (psql). Quelle: Eigene Darstellung

Es werden die Exemplare des Titels „Zwischen Tabelle und Zeile“ im Umkreis von 600 km um Standort 3 in Berlin, die nicht verliehen sind, gesucht. Die Entfernung folgt dem sphärischen Kosinussatz mit dem mittleren Erdradius von 6371 km (Veness, 2022). Die Vorauswahl über den Breitengrad, ein Grad entspricht rund 111,2 km, kann den Index *idx_standort_koordinaten* nutzen und verkleinert die Zahl der Entfernungsberechnungen (Veness, 2020).

Abfrage 2: Exemplare mit Besitzer und Abholort

2_Erarbeitung : psql — Konsole

File Edit View Bookmarks Plugins Settings Help

```
buchtausch=> SELECT e.ExemplarID, b.Titel, n.Vorname || ' ' || n.Name AS Besitzer,
       s.Straße || ' ' || s.Hausnummer AS Abholort, s.Ort
  FROM Exemplar e
  JOIN Buchtitel b ON b.BuchtitelID = e.BuchtitelID
  JOIN Nutzer n    ON n.NutzerID    = e.BesitzerNutzerID
  JOIN Standort s  ON s.NutzerID    = e.BesitzerNutzerID AND s.StandortID = e.StandortID
 ORDER BY e.ExemplarID;
 exemplarid |              titel              |     besitzer     |       abholort        |   ort
------------+---------------------------------+------------------+-----------------------+---------
          1 | Das Testroman-Experiment        | Anna Musterfrau  | Musterstraße 1        | Berlin
          2 | Beispiel-Drama in drei Akten    | Ben Mustermann   | Beispielweg 2         | Berlin
          3 | Der Fall Mustermann             | Clara Beispiel   | Testallee 3           | Berlin
          4 | Die Reise ins Datenland         | David Testmann   | Probegasse 4          | Berlin
          5 | Zwischen Tabelle und Zeile      | Elisa Probe      | Demoplatz 5           | München
          6 | Drache Dummy und der Probestein | Fabian Demo      | Dummyring 6           | München
          7 | Das Kinderbuch der Beispiele    | Greta Dummy      | Platzhalterstraße 7   | München
          8 | Kurze Geschichte der Testdaten  | Hans Platzhalter | Vorlagenweg 8         | München
          9 | Planet Placeholder              | Ida Vorlage      | Stichprobenallee 9    | Hamburg
         10 | Handbuch der Beispieldatenbank  | Jonas Stichprobe | Attrappenstraße 10    | Hamburg
         11 | Das Testroman-Experiment        | Kira Attrappe    | Exempelufer 11        | Hamburg
         12 | Der Fall Mustermann             | Leon Exempel     | Datenstraße 12        | Hamburg
         13 | Beispiel-Drama in drei Akten    | Anna Musterfrau  | Zweitwohnungsweg 13   | München
         14 | Zwischen Tabelle und Zeile      | Ben Mustermann   | Nebenstandortgasse 14 | Hamburg
(14 rows)

buchtausch=>
```

Abbildung 16: Ausgabe der Abfrage 2 (psql). Quelle: Eigene Darstellung

Ein Join über die erste Dreifachbeziehung. *Exemplar* verbindet Buchtitel, Nutzer als Besitzer und Standort als Abholort.

17 Abfrage 3

Abfrage 3: Ausleihen mit Beteiligten und Zeitslot

Ein Join über die zweite Dreifachbeziehung. *Ausleihe* verbindet Exemplar, Zeitslot, Ausleiher und Verleiher. Die beiden Nutzer werden über zwei Aliase aufgelöst, der Zeitslot über den zusammengesetzten Schlüssel aus Exemplar und Zeitslot. Zwei Ausleihen sind noch nicht zurückgegeben, ihr Rückgabedatum ist leer.

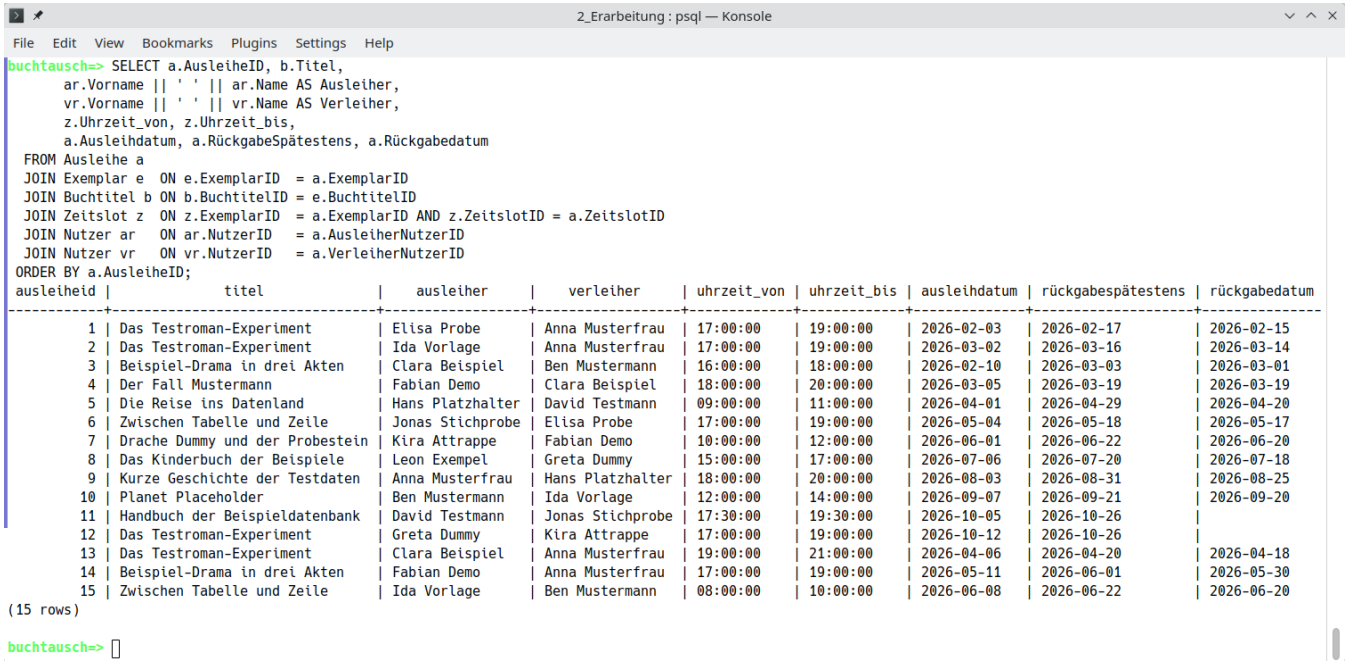


Abbildung 17: Ausgabe der Abfrage 3 (psql). Quelle: Eigene Darstellung

18 Abfragen 4 und 5

Abfrage 4: Bewertungen mit Titel und Beteiligten

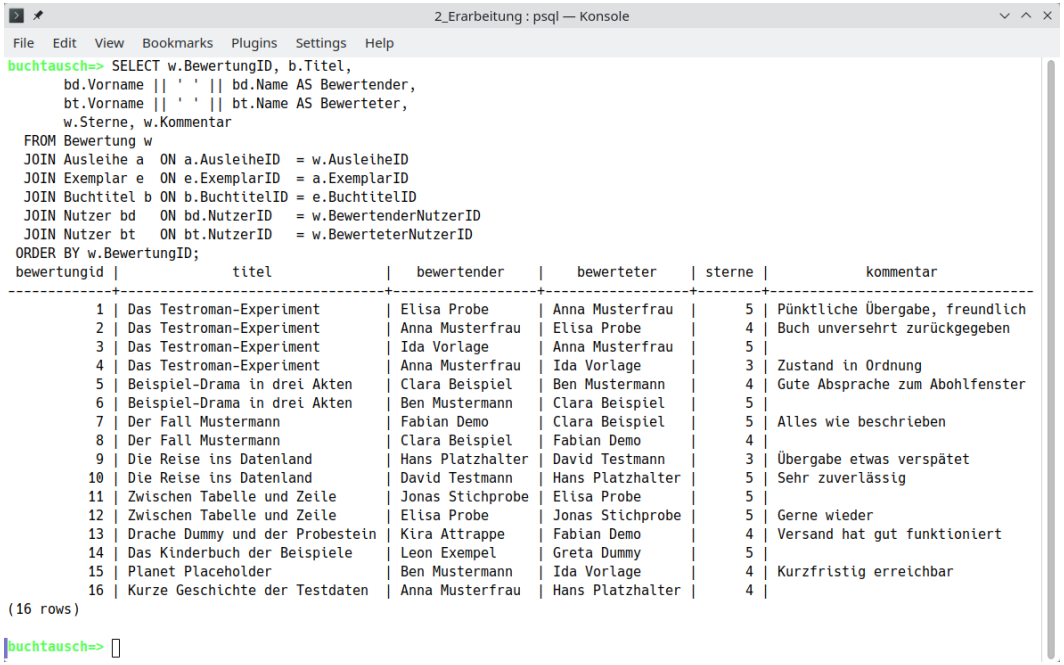


Abbildung 18: Ausgabe der Abfrage 4 (psql). Quelle: Eigene Darstellung

Ein Join über die dritte Dreifachbeziehung. *Bewertung* verbindet die Ausleihe mit dem bewertenden und dem bewerteten Nutzer. Der Titel ergibt sich über Ausleihe und Exemplar.

Abfrage 5: Durchschnittliche Bewertung je Nutzer

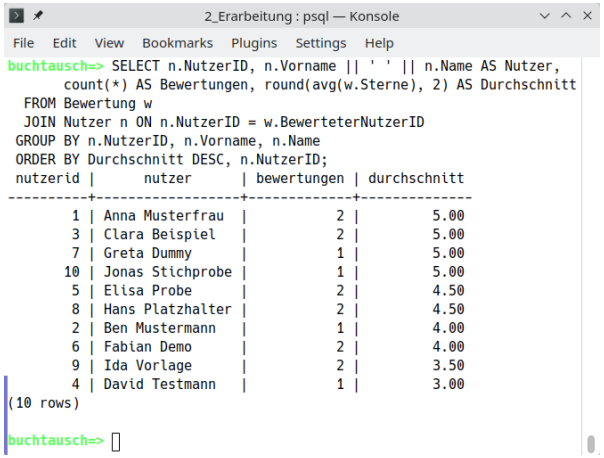


Abbildung 19: Ausgabe der Abfrage 5 (psql). Quelle: Eigene Darstellung

Eine Auswertung der wechselseitigen Bewertungen. Gruppiert wird nach dem bewerteten Nutzer, sortiert nach dem Durchschnitt.

Literaturverzeichnis

Veness, C. (2020). *Selecting points from a database by latitude/longitude within a bounding circle*. Movable Type Scripts. https://www.movable-type.co.uk/scripts/latlong-db.html

Veness, C. (2022). *Calculate distance, bearing and more between latitude/longitude points*. Movable Type Scripts. https://www.movable-type.co.uk/scripts/latlong.html